



AI Evaluation & Benchmarking

AICB-P1 · Ngày 14 · Đo lường chất lượng AI một cách khoa học

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Sếp hỏi: AI agent của mình tốt hơn ChatGPT bao nhiêu? Bạn nói sao nếu không có benchmark?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Từ Monitoring sang Evaluation
2. Traditional Metrics & Paradigm Shift
3. Evaluation fundamentals
4. Metrics cho AI agent
5. Benchmark design
6. LLM-as-Judge
7. Evaluation Frameworks: RAGAS, DeepEval, TruLens
8. So sánh 3 frameworks
9. Failure analysis, Lab 14 + deliverable

- Hiểu vì sao **evaluation là engineering discipline**, không phải cảm tính
- Thiết kế **benchmark** với golden dataset, edge cases, và stratified sampling
- Sử dụng **LLM-as-Judge** để đánh giá output tự động với rubric và scale rõ ràng
- Chạy evaluation với **3 frameworks**: RAGAS, DeepEval, TruLens — hiểu trade-offs
- Thực hiện **failure analysis** và xây dựng continuous improvement loop

Evaluation report cho agent gồm benchmark 20 test cases, framework scores, LLM-as-Judge results, failure analysis, và improvement recommendations

- 1 golden dataset: 20 question-answer pairs với expected answers
- 1 evaluation chạy bằng ít nhất 1 framework (RAGAS hoặc DeepEval hoặc TruLens)
- 1 LLM-as-Judge scoring: rubric 1–5 cho ít nhất 10 responses
- 1 failure analysis: 3 worst cases với root cause và fix recommendations

Từ Monitoring Sang Evaluation

Day 13 cho biết agent đang chạy thế nào. Day 14 cho biết agent chạy **tốt** đến đâu — hai câu hỏi khác nhau

Day 13 đã làm được

- Structured logging + correlation ID
- Langfuse/LangSmith tracing
- Dashboard: latency, cost, errors
- Alert rules cho thresholds

Day 14 sẽ trả lời

- Agent trả lời **đúng** không?
- Output **faithful** với context không?
- Khi sai, sai **ở đâu** và **tại sao**?
- Agent **tốt hơn hay tệ hơn** tuần trước?

Langfuse traces từ Day 13 = **data source cho evaluation**. Dùng logged outputs để chạy RAGAS/DeepEval offline, không cần re-run agent.

Traditional Metrics: Tại Sao Không Đủ?

Từ Exact Match đến Semantic Evaluation — hiểu quá khứ để đánh giá đúng AI tạo sinh

ML Truyền thống (Tất định)

- Output cố định: Đúng hoặc Sai
- Exact Match là tiêu chuẩn vàng
- Precision, Recall, F1 Score
- Confusion Matrix rõ ràng

Generative AI (Xác suất)

- Output là phân phối trên không gian từ vựng
- Có hàng ngàn cách diễn đạt cùng 1 ý nghĩa
- Exact Match = 0 dù ngữ nghĩa đúng 100%
- Cần evaluation mới cho paradigm mới

Các thước đo truyền thống **trừng phạt sự biến đổi phong cách và bỏ sót mâu thuẫn logic**. Chúng ta cần cách đánh giá khác.

BLEU

- Dịch máy (Precision)
- Đo n-gram overlap
- Brevity penalty

ROUGE

- Tóm tắt (Recall)
- ROUGE-N, ROUGE-L (LCS)
- Từ không cần liên tiếp

WER

- Speech / edit distance
- Càng thấp càng tốt
- Đòi hỏi sao chép 1:1

Ví dụ thất bại:

Ground Truth: “Hà Nội là thủ đô.” Agent: “Thủ đô của Việt Nam là Hà Nội.”

→ Exact Match = 0, F1 token-level giảm mạnh — **dù ngữ nghĩa đúng 100%**

N-gram metrics trừng phạt paraphrasing và bỏ sót mâu thuẫn logic (“nên hoàn tiền” vs “không nên hoàn tiền” = ROUGE cao!)

Cosine Similarity

- Embedding model ánh xạ câu thành vector $\mathbb{R}^d$
- Câu cùng ý nghĩa $\rightarrow$ vector gần nhau
- $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$
- 1.0 = giống hệt, 0.0 = không liên quan

Giới hạn: Cosine bắt Topic tốt nhưng bỏ sót Contradiction
("yêu" vs "ghét" Apple Vision Pro $\rightarrow$ cosine $>$ 0.90)

1. N-gram: fail ở paraphrase
2. Cosine: fail ở contradiction
3. $\Rightarrow$ Cần LLM hiểu logic & ngữ nghĩa sâu

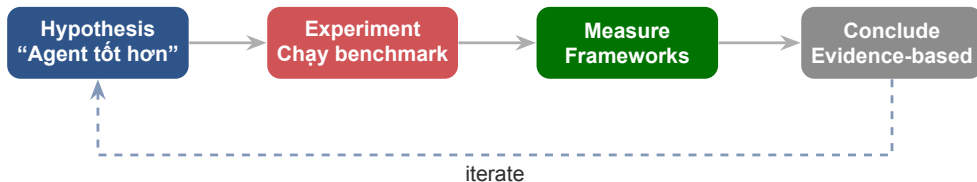
Hành trình:

Exact Match $\rightarrow$ N-gram $\rightarrow$ Embedding $\rightarrow$
LLM-as-Judge

Evaluation Fundamentals

“Cảm thấy agent trả lời tốt” không phải evidence. Evaluation biến cảm nhận thành số liệu có thể so sánh, lặp lại, và cải thiện

03



Không đo = không cải thiện. Evaluation phải **lặp lại được**, **so sánh được**, và **chạy tự động được**.

Batch test trên golden dataset.

Khi nào: mỗi release, mỗi prompt change

Tool: RAGAS, DeepEval, TruLens

Monitor quality trên production.

Khi nào: continuous, real traffic

Tool: TruLens, Langfuse

Expert review sampled outputs.

Khi nào: weekly, high-stakes

Tool: annotation UI, spreadsheet

Lưu ý: Chỉ offline eval = không biết production quality. Chỉ human eval = không scale. Cần **kết hợp cả 3**.

Trigger	Loại eval	Mục tiêu
Mỗi code release	Offline (full suite)	Regression check
Mỗi prompt change	Offline (targeted)	Không làm hỏng chỗ khác
Weekly	Human (sampled)	Quality trend
Continuous	Online (monitoring)	Catch degradation
Trước demo/launch	Offline + Human	Confidence trước stakeholders

Eval nên chạy **tự động trong CI/CD**. Agent không pass eval = không được deploy, giống unit test.

Metrics Cho AI Agent

Không phải mọi metric đều quan trọng như nhau — chọn metrics phải gắn với use case và business outcome

04

Task Completion

- Binary: đúng hay sai?
- Partial credit: đúng bao nhiêu %?
- Steps completed: hoàn thành bao nhiêu bước?

RAG-Specific

- Faithfulness: dựa trên context?
- Answer relevancy: trả lời đúng câu hỏi?
- Context recall: retrieve đủ evidence?
- Context precision: context có liên quan?

Answer Quality

- Accuracy: thông tin đúng không?
- Completeness: đủ chi tiết chưa?
- Coherence: mạch lạc, dễ hiểu?
- Citation accuracy: trích đúng nguồn?

Business

- User satisfaction (thumbs up/down)
- Time saved per interaction
- Cost per interaction
- Adoption rate over time



Context Recall thấp = retrieve thiếu. Context Precision thấp = retrieve thừa. Faithfulness thấp = hallucinate. Answer Relevancy thấp = trả lời lạc đề.

Benchmark Design

Evaluation tốt bao nhiêu phụ thuộc vào benchmark tốt bao nhiêu
— garbage in, garbage out

05

Golden dataset gồm

- Expected answers do expert viết
- Cover tất cả use cases chính
- Có difficulty levels: easy, medium, hard
- Có edge cases và adversarial inputs

Nếu expected answer sai, toàn bộ evaluation sẽ misleading.

Rule: ít nhất 2 experts review mỗi answer.

Benchmark Sizing

- **30–50 Qs:** “Smoke Test” — bắt showstoppers, rapid iteration, feedback trong phút
- **100–200 Qs:** “Statistical Significance” — giảm noise, cover long-tail, detect subtle regressions (5% error)

30–50	Rapid dev, catch major breaks
100–200	Stable system, need confidence

Lưu ý: 20 cho lab. 30–50 cho rapid dev. 100–200 cho production. Dưới 30 = quá ít

- **5 Easy:** factual lookup, single-doc
- **7 Medium:** multi-step reasoning, 2–3 docs
- **5 Hard:** complex/ambiguous, nhiều cách hiểu
- **3 Adversarial:** out-of-scope, cố tình phá

- question: câu hỏi user
- ground_truth: expected answer
- difficulty: easy/medium/hard
- category: use case type
- source_docs: documents cần retrieve

Bắt đầu từ **real user questions** trong Langfuse logs (Day 13). Synthetic questions dễ tạo nhưng không phản ánh reality.

- Ambiguous queries (nhiều cách hiểu)
- Out-of-scope (ngoài domain)
- Adversarial (cố tình phá)
- Multilingual (VN + EN mixed)
- Long context (nhiều tài liệu)

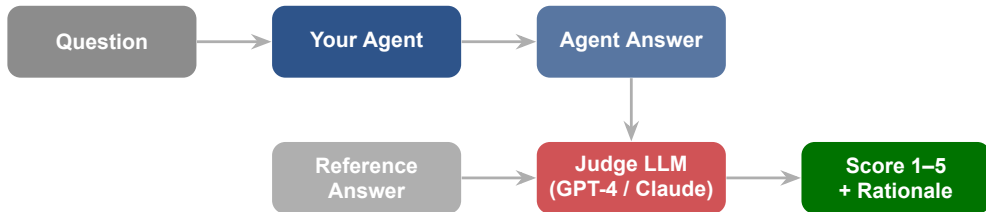
- Proportional cho mỗi use case
- Đủ samples cho mỗi difficulty level
- Đại diện các user types khác nhau
- Cân bằng giữa happy path và edge case

Benchmark phải **evolve**. Thêm failure cases vào benchmark mỗi sprint. Track changes trong Git để tránh data contamination.

LLM-as-Judge

Human eval chính xác nhất nhưng không scale. LLM-as-Judge cho phép đánh giá hàng trăm outputs tự động với rubric rõ ràng

06



Judge LLM nhận question + agent answer + reference answer + rubric, rồi cho điểm kèm giải thích. Scale tốt hơn human review.


```
JUDGE_PROMPT = """"
Score the answer on a scale of 1-5:
5 = Correct, complete, well-cited
4 = Mostly correct, minor gaps
3 = Partially correct, some errors
2 = Significant errors or missing info
1 = Wrong or irrelevant

Question: {question}
Reference: {reference}
Agent answer: {answer}

Score (1-5):
Rationale:
""""
```

Rubric tốt = tiêu chí cụ thể + examples cho mỗi mức. Rubric mơ hồ sẽ cho scores không nhất quán.

```
from openai import OpenAI

client = OpenAI()

def judge_response(question, reference, answer):
    prompt = JUDGE_PROMPT.format(
        question=question, reference=reference, answer=answer
    )
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0 # deterministic scoring
    )
    output = response.choices[0].message.content
    score = int(output.split("Score")[-1].strip()[0])
    rationale = output.split("Rationale:")[-1].strip()
    return {"score": score, "rationale": rationale}
```

temperature=0 cho consistent scoring. Chạy 2–3 lần để check variance.

Judge ưu tiên answer xuất hiện trước.

Fix: randomize thứ tự

Judge cho điểm cao hơn answer dài hơn.

Fix: rubric nói rõ “concise is OK”

GPT-4 judge ưu tiên GPT-4 output.

Fix: dùng judge khác model

Lưu ý: LLM-as-Judge không hoàn hảo. Cần **calibrate against human**: so sánh scores của judge với expert ratings trên 50+ samples.

- ☒ **Multiple judges:** dùng 2–3 LLMs khác nhau, lấy majority vote hoặc average
- ☒ **Randomize order:** đổi vị trí answer A/B giữa các lần chạy
- ☒ **Include rationale:** yêu cầu judge giải thích điểm, không chỉ cho số
- ☒ **Calibrate:** so sánh judge scores với human ratings trên subset
- ☐ **Đừng tin judge 100%.** LLM judge vẫn sai, đặc biệt ở domain chuyên biệt

Viết rubric scoring 1–5 cho agent của nhóm bạn. Share với người bên cạnh và so sánh.

Template

- 5 = _____
- 4 = _____
- 3 = _____
- 2 = _____
- 1 = _____

Gợi ý tiêu chí

- Correctness (đúng sự thật?)
- Completeness (đủ chi tiết?)
- Relevance (trả lời đúng câu hỏi?)
- Citation (trích nguồn?)
- Tone (giọng phù hợp context?)

Evaluation Frameworks: RA-GAS, DeepEval, TruLens

3 frameworks phổ biến nhất cho AI/RAG evaluation — mỗi cái có thể mạnh riêng, chọn theo use case

```
# Cài đặt tất cả frameworks + dependencies
pip install ragas deepeval trulens datasets openai

# Hoặc cài riêng từng cái:
pip install ragas>=0.2.0      # RAG evaluation
pip install deepeval>=1.0     # LLM unit testing
pip install trulens>=1.0      # Feedback functions

# Kiểm tra
python -c "import ragas; import deepeval; import trulens; print('OK')"
```

Lưu ý: Cần OpenAI API key (hoặc Azure) vì cả 3 frameworks dùng LLM để tính metrics. Set OPENAI_API_KEY trước khi chạy.

RAGAS

RAG-focused
metrics

Best for: RAG pipeline
evaluation with
standardized metrics

DeepEval

Unit testing
for LLMs

Best for: CI/CD
integration, pytest-like
LLM testing

TruLens

Feedback
functions

Best for: production
monitoring +
offline eval combo

Core Metrics

- **Faithfulness:** answer dựa trên context?
- **Answer Relevancy:** trả lời đúng question?
- **Context Recall:** retrieve đủ evidence?
- **Context Precision:** context relevant?

Đặc điểm

- Open source, community lớn
- Tập trung 100% vào RAG
- Không cần ground truth cho một số metrics
- Tích hợp LangChain, LlamaIndex

Agent chủ yếu là RAG pipeline. Cần metrics chuẩn hóa để so sánh qua thời gian.
Team quen Python + HuggingFace ecosystem.

```
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import (
    Faithfulness, AnswerRelevancy,
    ContextRecall, ContextPrecision
)

# Step 1: Chuẩn bị dataset (format bat buoc)
eval_data = {
    "question": ["What is refund policy?", "How to cancel?"],
    "answer": [agent_answer_1, agent_answer_2],
    "contexts": [[ctx_chunk_1, ctx_chunk_2], [ctx_chunk_3]],
    "ground_truth": ["Refund within 30 days...", "Cancel via..."]
}
dataset = Dataset.from_dict(eval_data)

# Step 2: Chạy evaluation (RAGAS v0.2+)
result = evaluate(dataset, metrics=[
    Faithfulness(), AnswerRelevancy(),
    ContextRecall(), ContextPrecision()
])
print(result) # (faithfulness: 0.82, answer_relevancy: 0.91, ...)
```

Core Metrics

- **G-Eval:** LLM-based scoring tùy chỉnh
- **Hallucination:** detect bịa thông tin
- **Answer Relevancy:** tương tự RAGAS
- **Toxicity:** phát hiện nội dung có hại
- **Bias:** phát hiện thiên lệch

Đặc điểm

- Pytest-native: `deepeval test run`
- CI/CD first design
- 14+ built-in metrics
- Custom metrics dễ tạo
- Dashboard web miễn phí (Confident AI)

Team muốn eval như unit test — pytest + assertions. Cần nhiều metric types (RAG + safety + custom). Muốn CI/CD integration nhanh nhất.

```
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import (
    FaithfulnessMetric, AnswerRelevancyMetric,
    HallucinationMetric
)

# Tao test case
test_case = LLMTestCase(
    input="What is refund policy?",
    actual_output=agent_answer,
    retrieval_context=[ctx_chunk_1, ctx_chunk_2],
    expected_output="Refund within 30 days..."
)

# Chay nhu unit test (threshold = 0.7)
faithfulness = FaithfulnessMetric(threshold=0.7)
hallucination = HallucinationMetric(threshold=0.5)

assert_test(test_case, [faithfulness, hallucination])
# PASSED neu score >= threshold, FAILED neu khong
```

Core Concepts

- **Feedback functions:** modular eval units
- **Groundedness:** answer grounded in context?
- **Relevance:** question-answer alignment
- **Comprehensiveness:** answer completeness
- **Custom feedback:** tự định nghĩa

Đặc điểm

- Instrument app **trực tiếp** (wrap decorator)
- Online + Offline eval trong 1 tool
- Dashboard built-in (Streamlit)
- Track experiments qua thời gian
- By Snowflake (acquired Truera 2024)

Cần **production monitoring + evaluation** trong cùng 1 tool. Muốn instrument trực

```
from trulens.core import TruSession, Feedback
from trulens.providers.openai import OpenAI as TruOpenAI
from trulens.apps.custom import TruCustomApp, instrument

provider = TruOpenAI() # LLM-based feedback

# Define feedback functions
f_groundedness = Feedback(provider.groundedness_measure_with_cot_reasons
    ).on_input_output()
f_relevance = Feedback(provider.relevance
    ).on_input_output()

# Instrument your app
tru_app = TruCustomApp(
    app=my_agent,
    app_name="RAG Agent v1",
    feedbacks=[f_groundedness, f_relevance]
)

with tru_app as recording:
    response = my_agent.query("What is refund policy?")
# Scores auto-logged to TruLens dashboard
```

So Sánh 3 Frameworks

Không có framework “tốt nhất” — chỉ có framework phù hợp nhất với team, use case, và workflow của bạn

08

Tiêu chí	RAGAS	DeepEval	TruLens
Focus	RAG metrics	LLM unit testing	Production monitoring
Metrics	4 core RAG	14+ (RAG + safety)	Feedback functions
CI/CD	Script-based	Pytest-native	Script-based
Production	Cần kết hợp tool khác	Cần kết hợp tool khác	Built-in dashboard
Custom metrics	Limited	Dễ, class-based	Dễ, function-based
Learning curve	Thấp	Thấp–Trung bình	Trung bình

- Agent = RAG pipeline
- Cần metrics chuẩn hóa
- So sánh across time
- Team nhỏ, cần nhanh
- Đã dùng LangChain

- Cần CI/CD assertions
- eval = test suite
- Cần safety metrics
- Team quen pytest
- Nhiều metric types

- Cần online + offline
- Track trends over time
- Instrument trực tiếp
- Dashboard built-in
- Enterprise needs

Nhiều team dùng **kết hợp**: RAGAS cho offline CI/CD metrics + TruLens cho production monitoring. DeepEval thay thế được RAGAS nếu cần thêm safety metrics.

Approach	Cost	Speed	Accuracy	Khi nào dùng
RAGAS	Thấp	Nhanh	Medium	RAG CI/CD
DeepEval	Thấp	Nhanh	Medium	General CI/CD
TruLens	Trung bình	Nhanh	Medium	Production
LLM-as-Judge	Trung bình	Nhanh	Med–High	Custom rubrics
Human eval	Cao	Chậm	Cao nhất	High-stakes, calibrate

Lưu ý: Automated eval (frameworks) cho **speed + scale**. Human eval cho **accuracy + trust**.
Production cần cả hai.

Score	Ý nghĩa	Action	Priority
0.8 – 1.0	Good	Monitor, maintain	Low
0.6 – 0.8	Needs work	Analyze failures, iterate	Medium
< 0.6	Significant issues	Deep investigation required	High

Framework + CI/CD = **quality gate** tự động. Agent với faithfulness < 0.7 sẽ không được deploy, giống failed unit test.

DeepEval: `deepeval test run test_eval.py` trong GitHub Actions.

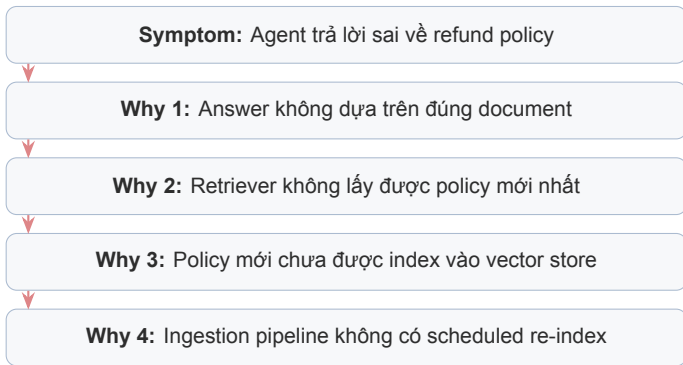
RAGAS/TruLens: custom script + threshold check.

Failure Analysis, Lab & Take-aways

Evaluation cho biết điểm số. Failure analysis cho biết tại sao điểm thấp. Lab cho bạn practice cả hai

Failure type	Triệu chứng	Root cause thường gặp
Wrong Answer	Trả lời sai sự thật	Retrieval miss, prompt ambiguous
Hallucination	Bịa thông tin không có trong context	Faithfulness guardrail yếu
Tool Failure	Tool gọi lỗi hoặc timeout	API down, wrong params
Refusal	Từ chối trả lời khi nên trả lời	Guardrails quá chặt
Slow	Response quá chậm	LLM model quá lớn, context dài

Phân loại failure **trước khi fix**. Nếu 80% failures là retrieval miss, thì fix retriever sẽ hiệu quả hơn fix prompt.



Vấn đề thật không phải prompt hay model. Vấn đề là **data pipeline**. Fix đúng chỗ sẽ giải quyết hàng loạt failures tương tự.

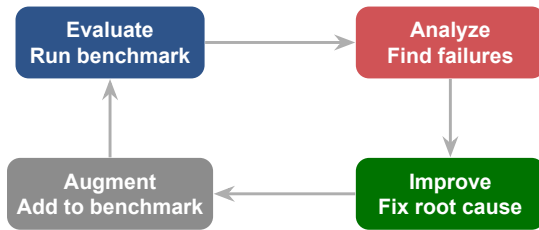
Cách làm

1. Collect tất cả failure cases
2. Group theo failure type
3. Trong mỗi type, cluster theo root cause
4. Prioritize: cluster lớn nhất fix trước

Fix 1 root cause giải quyết nhiều failures cùng lúc.

Ví dụ: fix retrieval indexing giải quyết 15/20 “wrong answer” cases.

Lưu ý: Đừng fix từng failure riêng lẻ. **Cluster rồi fix root cause** sẽ hiệu quả hơn nhiều lần.



Eval trước khi optimize. Fix dựa trên evidence. Thêm failure cases vào benchmark. Lặp lại.

Tạo benchmark cho agent, chạy evaluation với framework, phân tích failures, và đề xuất improvements dựa trên data.

1. Tạo golden dataset: 20 QA pairs (5 easy + 7 medium + 5 hard + 3 adversarial)
2. Chạy agent trên toàn bộ dataset, collect results
3. Evaluation với framework (chọn 1): RAGAS hoặc DeepEval hoặc TruLens
4. LLM-as-Judge: scoring 1–5 với rubric cho 10+ responses
5. Failure analysis: chọn 3 worst cases, 5 Whys cho mỗi case
6. Improvement log: ghi lại recommendations dựa trên root cause

Evaluation

- Golden dataset (20 QA pairs)
- Framework scores (ghi rõ dùng framework nào)
- LLM-as-Judge scores (10+ items)
- Score interpretation + so sánh

Failure Analysis

- 3 worst cases detailed
- 5 Whys per case
- Root cause clusters
- Improvement recommendations

Lưu ý: Không cần perfect scores. Điều cần chứng minh là bạn **biết agent tốt đến đâu, yếu ở đâu, và phải fix gì.**

Chạy cùng dataset trên 2 frameworks khác nhau, so sánh scores và rút ra insights về khác biệt giữa các frameworks.

Câu hỏi cần trả lời

- Scores có consistent không?
- Framework nào strict hơn?
- Failure cases có giống nhau không?

Expected insight

- Metrics tên giống nhưng tính khác
- DeepEval strict hơn vì threshold-based
- TruLens cho trend data bổ sung

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Evaluation là engineering discipline**, không phải cảm tính. “Cảm thấy tốt” không phải evidence — framework scores mới là.
- 2 **3 frameworks, 3 strengths**: RAGAS cho RAG metrics chuẩn, DeepEval cho CI/CD testing, TruLens cho production monitoring. Chọn theo workflow.
- 3 **LLM-as-Judge** bổ sung cho framework metrics — cho custom rubrics và nuanced scoring, nhưng cần calibrate.
- 4 **Failure analysis** là fastest path to improvement. Cluster failures, tìm root cause, fix systematic thay vì từng case.

Triển Khai Thực Tế & Định Hướng Chuyên Sâu

“15 ngày từ “AI là gì” đến agent deployed, monitored, evaluated. Tiếp theo: đi sâu theo hướng nào? ”

- Review toàn bộ artifacts từ Day 1–14: agent có gì, thiếu gì
- Suy nghĩ: bạn muốn đi sâu Business, Infra, hay Application track?
- Chuẩn bị câu hỏi cho AMA (Ask Me Anything) session cuối Phase 1

1. RAGAS — docs.ragas.io. DeepEval — docs.confident-ai.com. TruLens — trulens.org.
2. Zheng et al. (2023), *Judging LLM-as-a-Judge* — [arXiv:2306.05685](https://arxiv.org/abs/2306.05685). Bias types, rubric design, MT-Bench.
3. OpenAI Evals — github.com/openai/evals. Framework cho custom evaluation pipelines.

Hỏi & Đáp

Evaluation tốt nghĩa là bạn biết agent tốt đến đâu, yếu ở đâu, và phải fix gì tiếp theo.